

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Вычислительные сети и контроль безопасности в компьютерных сетях»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

«Настройка межсетевого экрана»

Вариант 3

Выполнили:

Дынина Е.А., студент группы N3349

Жук А.В., студент группы N3348

Исаева А., студент группы N3349

Проверил:

Есипов Д.А., ассистент ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

Содержание	3
Введение	4
Ход работы	5
1.1 Настройка лабораторного стенда.....	5
1.2 Настройка маршрутизации проходящего трафика на локальном сервере.....	10
1.3 Настройка межсетевого экрана	11
1.4 Тестирование.....	14
Заключение.....	18

ВВЕДЕНИЕ

Цель работы — изучение основных принципов работы межсетевых экранов и освоение базовой настройки правил iptables.

Для достижения поставленной цели необходимо решить следующие задачи:

- настроить лабораторный стенд;
- настроить маршрутизацию проходящего трафика на локальном сервере;
- заблокировать все входящие Telnet-соединения;
- установить блокировку для входящих запросов TCP, не открывающих новое соединение и не принадлежащих никакому из установленных соединений;
- ограничить количество параллельных соединений к серверу SSH для одного адреса;
- настроить систему так, чтобы ICMP пакеты принимались не более одного раза в секунду;
- протестировать работу выполненных настроек;
- результаты выполнения работы оформить в виде отчета.

ХОД РАБОТЫ

Межсетевой экран — технологический барьер, предназначенный для предотвращения несанкционированного или нежелательного сообщения между компьютерными сетями или хостами.

iptables — утилита командной строки, является стандартным интерфейсом управления работой межсетевого экрана (брандмауэра) netfilter для ядер Linux, начиная с версии 2.4. Для использования утилиты iptables требуются привилегии суперпользователя (root).

1.1 Настройка лабораторного стенда

Для начала соберем и настроим макет для работы (рис. 1).

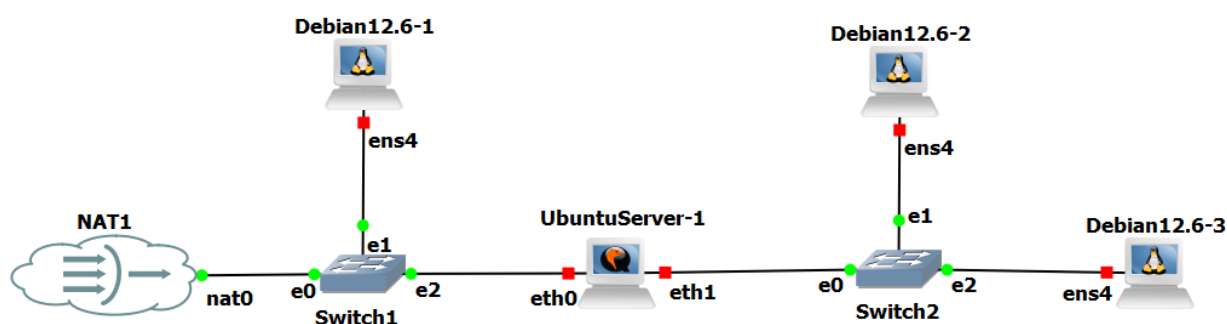


Рисунок 1 – Лабораторный стенд

Стенд состоит из нескольких клиентов, локального и удаленного сервера, свичей и роутера. Подробное описание всех устройств представлено в таблице 1.

Таблица 1 – Элементы макета и их назначение

Роль	Устройство в GNS3	Назначение
ExternalClient	Debian12.6-1 (ens4)	Клиент, отправляющий запросы из "внешней" сети.
InternalClient	Debian12.6-2 (ens4)	Дополнительный клиент во "внутренней" сети.
IPtablesFW (Локальный сервер/МЭ)	UbuntuServer-1 (eth0, eth1)	Межсетевой экран, фильтрующий трафик и выполняющий маршрутизацию.

Server-1 (Удаленный)	Debian12.6-3 (ens4)	Сервер, принимающий запросы.
Switch1	Switch2	Объединяет IPtablesFW, InternalClient и UbuntuServer-1.
Switch2	Switch1	Объединяет ExternalClient, IPtablesFW и NAT1.
NAT1	NAT1	Эмулятор домашнего роутера. Он автоматически раздает адреса (DHCP) и "подменяет" внутренний трафик, позволяя устройствам в лаборатории выходить в Интернет.

Для настройки IPtablesFW применим команду:

```
sudo nano /etc/netplan/00-installer-config.yaml
```

На рисунке 2 представлено то, что нужно прописать в файле.

```
GNU nano 7.2 /etc/netplan/00-installer-config.yaml
network:
  version: 2
  ethernet:
    ens3:
      addresses:
        - 10.10.10.1/24
      dhcp4: no
    ens4:
      addresses:
        - 192.168.1.1/24
      dhcp4: no
```

Рисунок 2 – Код для настройки IPtablesFW

После применяем изменений с помощью команды.

```
sudo netplan apply
```

На рисунке 3 можно увидеть, что получится в итоге.

```
root@u-live-srvr:~# ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 0c:37:e1:7f:00:00 brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 10.10.10.1/24 brd 10.10.10.255 scope global ens3
        valid_lft forever preferred_lft forever
    inet6 fe80::e37:e1ff:fe7f:0/64 scope link
        valid_lft forever preferred_lft forever
3: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 0c:37:e1:7f:00:01 brd ff:ff:ff:ff:ff:ff
    altname enp0s4
    inet 192.168.1.1/24 brd 192.168.1.255 scope global ens4
        valid_lft forever preferred_lft forever
    inet6 fe80::e37:e1ff:fe7f:1/64 scope link
        valid_lft forever preferred_lft forever
```

Рисунок 3 – Результат настройки IPtablesFW

На рисунках 4–7 показана настройка других компонент стенда.

```
debian@debian:~$ su -
Password:
root@debian:~# nano /etc/network/interfaces
GNU nano 7.2 /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# DHCP config for ens4
#auto ens4
#iface ens4 inet dhcp

# Static config for ens4
auto ens4
iface ens4 inet static
    address 10.10.10.10
    netmask 255.255.255.0
    gateway 10.10.10.1
#    dns-nameservers 192.168.1.1

root@debian:~# systemctl restart networking
root@debian:~# ping 10.10.10.1
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data.
64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=2.68 ms
64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=1.92 ms
64 bytes from 10.10.10.1: icmp_seq=3 ttl=64 time=1.46 ms
^C
--- 10.10.10.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2011ms
rtt min/avg/max/mdev = 1.461/2.020/2.681/0.503 ms
root@debian:~#
```

Рисунок 4 – Настройка ExternalClient

```

debian@debian:~$ su -
Password:
root@debian:~# nano /etc/network/interfaces
GNU nano 7.2 /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# DHCP config for ens4
#auto ens4
#iface ens4 inet dhcp

# Static config for ens4
auto ens4
iface ens4 inet static
    address 192.168.1.10
    netmask 255.255.255.0
    gateway 192.168.1.1
#    dns-nameservers 192.168.1.1

root@debian:~# systemctl restart networking
root@debian:~# ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=3.39 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=1.35 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=1.69 ms
64 bytes from 192.168.1.1: icmp_seq=4 ttl=64 time=1.90 ms
^C
--- 192.168.1.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3023ms
rtt min/avg/max/mdev = 1.353/2.083/3.394/0.781 ms
root@debian:~# ping 10.10.10.10
PING 10.10.10.10 (10.10.10.10) 56(84) bytes of data.
64 bytes from 10.10.10.10: icmp_seq=1 ttl=63 time=4.03 ms
64 bytes from 10.10.10.10: icmp_seq=2 ttl=63 time=2.77 ms
64 bytes from 10.10.10.10: icmp_seq=3 ttl=63 time=2.75 ms
^C
--- 10.10.10.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2015ms
rtt min/avg/max/mdev = 2.749/3.182/4.029/0.598 ms
root@debian:~# █

```

Рисунок 5 – Настройка InternalClient

```
debian@debian:~$ sudo passwd root
New password:
Retype new password:
passwd: password updated successfully
debian@debian:~$ su -
Password:
root@debian:~# nano /etc/network/interfaces
GNU nano 7.2 /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# DHCP config for ens4
#auto ens4
#iface ens4 inet dhcp

# Static config for ens4
auto ens4
iface ens4 inet static
    address 192.168.1.20
    netmask 255.255.255.0
    gateway 192.168.1.1
#    dns-nameservers 192.168.1.1

root@debian:~# systemctl restart networking
```

Рисунок 6 – Начало настройки Server-1


```

root@debian:~# ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=2.42 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=1.18 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=0.965 ms
^C
--- 192.168.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2022ms
rtt min/avg/max/mdev = 0.965/1.520/2.417/0.640 ms
root@debian:~# ping 192.168.1.10
PING 192.168.1.10 (192.168.1.10) 56(84) bytes of data.
64 bytes from 192.168.1.10: icmp_seq=1 ttl=64 time=2.11 ms
64 bytes from 192.168.1.10: icmp_seq=2 ttl=64 time=1.23 ms
64 bytes from 192.168.1.10: icmp_seq=3 ttl=64 time=0.864 ms
^C
--- 192.168.1.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2013ms
rtt min/avg/max/mdev = 0.864/1.400/2.110/0.523 ms
root@debian:~# ping 10.10.10.10
PING 10.10.10.10 (10.10.10.10) 56(84) bytes of data.
64 bytes from 10.10.10.10: icmp_seq=1 ttl=63 time=2.23 ms
64 bytes from 10.10.10.10: icmp_seq=2 ttl=63 time=1.90 ms
64 bytes from 10.10.10.10: icmp_seq=3 ttl=63 time=2.05 ms
^C
--- 10.10.10.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2016ms
rtt min/avg/max/mdev = 1.898/2.058/2.229/0.135 ms
root@debian:~# ping 10.10.10.1
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data.
64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=1.19 ms
64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=1.51 ms
64 bytes from 10.10.10.1: icmp_seq=3 ttl=64 time=1.37 ms
^C
--- 10.10.10.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2030ms
rtt min/avg/max/mdev = 1.185/1.356/1.510/0.133 ms
root@debian:~#

```

Рисунок 7 – Конец настройки Server-1

1.2 Настройка маршрутизации проходящего трафика на локальном сервере

Включим IP forwarding — функцию, позволяющую устройству принимать пакеты, предназначенные не ему, и перенаправлять их между различными сетевыми интерфейсами, — на IPtablesFW с помощью команд:

```

sudo sysctl net.ipv4.ip_forward=1
echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.conf

```

На рисунке 8–10 проверим произведенную настройку.

```

root@u-live-srvr:~# cat /proc/sys/net/ipv4/ip_forward
1
root@u-live-srvr:~# ip route show
10.10.10.0/24 dev ens3 proto kernel scope link src 10.10.10.1
192.168.1.0/24 dev ens4 proto kernel scope link src 192.168.1.1
root@u-live-srvr:~# _

```

Рисунок 8 – Проверка включения IP forwarding на IptablesFW и таблицы маршрутизации

```

root@u-live-srvr:~# sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination

```

Рисунок 9 – Просмотр правил в таблице filter

```

root@u-live-srvr:~# sudo iptables -t nat -L
Chain PREROUTING (policy ACCEPT)
target     prot opt source                destination

Chain INPUT (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination

Chain POSTROUTING (policy ACCEPT)
target     prot opt source                destination

```

Рисунок 10 – Просмотр правил в таблице NAT

1.3 Настройка межсетевого экрана

Для начала разрешим пакеты, направленные на сам фаервол и проходящие через транзитом, если они являются частью уже установленных соединений.

```

iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

```

```

root@u-live-srvr:~# sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
root@u-live-srvr:~# sudo iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
root@u-live-srvr:~# sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination            ctstate RELATED,ESTABLISHED
ACCEPT     all  --  anywhere              anywhere

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination            ctstate RELATED,ESTABLISHED
ACCEPT     all  --  anywhere              anywhere

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
root@u-live-srvr:~#

```

Рисунок 11 – Разрешение пакетов, являющихся частью установленного соединения

Правила применены к обеим цепочкам (INPUT и FORWARD) для обеспечения полной фильтрации трафика как направленного на сам межсетевой экран, так и проходящего через него транзитом.

Перейдем к первому пункту варианта 3 и заблокируем все входящие Telnet-соединения с адреса 10.10.10.10 (рис. 12).

```

# Блокируем Telnet с 10.10.10.10 на САМ фаервол
sudo iptables -A INPUT -s 10.10.10.10 -p tcp --dport 23 -j DROP

```

```

# Блокируем Telnet с 10.10.10.10 ЧЕРЕЗ фаервол к серверам
sudo iptables -A FORWARD -s 10.10.10.10 -p tcp --dport 23 -j DROP

```

```

root@u-live-srvr:~# sudo iptables -A INPUT -s 10.10.10.10 -p tcp --dport 23 -j DROP
root@u-live-srvr:~# sudo iptables -A FORWARD -s 10.10.10.10 -p tcp --dport 23 -j DROP
root@u-live-srvr:~# sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination            ctstate RELATED,ESTABLISHED
ACCEPT     all  --  anywhere              anywhere
DROP       tcp  --  10.10.10.10          anywhere              tcp dpt:telnet

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination            ctstate RELATED,ESTABLISHED
ACCEPT     all  --  anywhere              anywhere
DROP       tcp  --  10.10.10.10          anywhere              tcp dpt:telnet

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination

```

Рисунок 12 – Блокировка входящих Telnet-соединений

Далее установим блокировку для входящих запросов TCP не открывающих новое соединение и не принадлежащих никакому из установленных соединений (рис. 13).

```

sudo iptables -A INPUT -p tcp -m conntrack --ctstate INVALID -j DROP
sudo iptables -A FORWARD -p tcp -m conntrack --ctstate INVALID -j DROP

```

```

root@u-live-srvr:~# sudo iptables -A INPUT -p tcp -m conntrack --ctstate INVALID -j DROP
root@u-live-srvr:~# sudo iptables -A FORWARD -p tcp -m conntrack --ctstate INVALID -j DROP
root@u-live-srvr:~# sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination              ctstate
ACCEPT     all  --  anywhere              anywhere                  ctstate RELATED,ESTABLISHED
DROP       tcp  --  10.10.10.10           anywhere                  tcp dpt:telnet
DROP       tcp  --  anywhere              anywhere                  ctstate INVALID
DROP       tcp  --  anywhere              anywhere                  ctstate INVALID

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination              ctstate
ACCEPT     all  --  anywhere              anywhere                  ctstate RELATED,ESTABLISHED
DROP       tcp  --  10.10.10.10           anywhere                  tcp dpt:telnet
DROP       tcp  --  anywhere              anywhere                  ctstate INVALID

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
root@u-live-srvr:~# sudo iptables -D INPUT 3
root@u-live-srvr:~# sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination              ctstate
ACCEPT     all  --  anywhere              anywhere                  ctstate RELATED,ESTABLISHED
DROP       tcp  --  10.10.10.10           anywhere                  tcp dpt:telnet
DROP       tcp  --  anywhere              anywhere                  ctstate INVALID

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination              ctstate
ACCEPT     all  --  anywhere              anywhere                  ctstate RELATED,ESTABLISHED
DROP       tcp  --  10.10.10.10           anywhere                  tcp dpt:telnet
DROP       tcp  --  anywhere              anywhere                  ctstate INVALID

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
root@u-live-srvr:~#

```

Рисунок 13 – Блокировка входящих TCP-запросов

Ограничим количество параллельных соединений к серверу SSH для одного адреса — не более 3 соединений одновременно (рис. 14).

```

sudo iptables -A INPUT -p tcp --dport 22 -m connlimit --connlimit-above 3 --connlimit-mask 32 -j DROP
sudo iptables -A FORWARD -p tcp --dport 22 -m connlimit --connlimit-above 3 -connlimit-mask 32 -j DROP

```

```

root@u-live-srvr:~# sudo iptables -A INPUT -p tcp --dport 22 -m connlimit --connlimit-above 3 --connlimit-mask 32 -j DROP
root@u-live-srvr:~# sudo iptables -A FORWARD -p tcp --dport 22 -m connlimit --connlimit-above 3 --connlimit-mask 32 -j DROP
root@u-live-srvr:~# sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination              ctstate
ACCEPT     all  --  anywhere              anywhere                  ctstate RELATED,ESTABLISHED
DROP       tcp  --  10.10.10.10           anywhere                  tcp dpt:telnet
DROP       tcp  --  anywhere              anywhere                  ctstate INVALID
DROP       tcp  --  anywhere              anywhere                  tcp dpt:ssh #conn src/32 > 3

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination              ctstate
ACCEPT     all  --  anywhere              anywhere                  ctstate RELATED,ESTABLISHED
DROP       tcp  --  10.10.10.10           anywhere                  tcp dpt:telnet
DROP       tcp  --  anywhere              anywhere                  ctstate INVALID
DROP       tcp  --  anywhere              anywhere                  tcp dpt:ssh #conn src/32 > 3

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination

```

Рисунок 14 – Ограничение количества параллельных соединений к серверу SSH

Нам было необходимо сделать так, чтобы ICMP пакеты типа echo-request принимались не более одного раза в секунду (рис. 15).

```
sudo iptables -A INPUT -p icmp --icmp-type echo-request -m limit --limit 1/sec --limit-burst 1 -j ACCEPT
sudo iptables -A INPUT -p icmp --icmp-type echo-request -j DROP
```

```
sudo iptables -A FORWARD -p icmp --icmp-type echo-request -m limit --limit 1/sec --limit-burst 1 -j ACCEPT
sudo iptables -A FORWARD -p icmp --icmp-type echo-request -j DROP
```

```
root@u-live-srvr:~# sudo iptables -A INPUT -p icmp --icmp-type echo-request -m limit --limit 1/sec --limit-burst 1 -j ACCEPT
root@u-live-srvr:~# sudo iptables -A INPUT -p icmp --icmp-type echo-request -j DROP
root@u-live-srvr:~#
root@u-live-srvr:~# sudo iptables -A FORWARD -p icmp --icmp-type echo-request -m limit --limit 1/sec --limit-burst 1 -j ACCEPT
root@u-live-srvr:~# sudo iptables -A FORWARD -p icmp --icmp-type echo-request -j DROP
root@u-live-srvr:~#
root@u-live-srvr:~# sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination              ctstate RELATED,ESTABLISHED
ACCEPT     all  --  anywhere               anywhere                  tcp dpt:telnet
DROP       tcp  --  10.10.10.10            anywhere                  ctstate INVALID
DROP       tcp  --  anywhere               anywhere                  tcp dpt:ssh #conn src/32 > 3
ACCEPT     icmp --  anywhere               anywhere                  icmp echo-request limit: avg 1/sec burst 1
DROP       icmp --  anywhere               anywhere                  icmp echo-request

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination              ctstate RELATED,ESTABLISHED
ACCEPT     all  --  anywhere               anywhere                  tcp dpt:telnet
DROP       tcp  --  10.10.10.10            anywhere                  ctstate INVALID
DROP       tcp  --  anywhere               anywhere                  tcp dpt:ssh #conn src/32 > 3
ACCEPT     icmp --  anywhere               anywhere                  icmp echo-request limit: avg 1/sec burst 1
DROP       icmp --  anywhere               anywhere                  icmp echo-request

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
```

Рисунок 15 – Ограничение приема ICMP пакетов по времени

И в конце сохраним правила (рис. 16).

```
iptables-save > /etc/iptables/rules.v4
```

```
root@u-live-srvr:~# sudo mkdir -p /etc/iptables
root@u-live-srvr:~# sudo iptables-save > /etc/iptables/rules.v4
root@u-live-srvr:~#
root@u-live-srvr:~# ls -la /etc/iptables/rules.v4
-rw-r--r-- 1 root root 1065 Apr  1 09:31 /etc/iptables/rules.v4
root@u-live-srvr:~#
```

Рисунок 16 – Сохранение правил

1.4 Тестирование

В рамках тестирования попробуем исправление правил: удаление первого прописанного правила для проверки корректности других правил. Например, при тестировании 4 правила ограничения ICMP-трафика было выявлено, что модуль conntrack классифицирует последующие пакеты одного ping-сеанса как ESTABLISHED, из-за чего они проходят фильтрацию до применения правила с лимитом. В рабочей конфигурации данные правила должны оставаться активными для обеспечения нормальной работы сетевых сервисов.

```

root@u-live-srvr:~# sudo iptables -D INPUT 1
root@u-live-srvr:~# sudo iptables -D FORWARD 1
root@u-live-srvr:~# sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination              tcp dpt:telnet
DROP       tcp  --  10.10.10.10            anywhere                  ctstate INVALID
DROP       tcp  --  anywhere               anywhere                  tcp dpt:ssh #conn src/32 > 3
ACCEPT     icmp --  anywhere               anywhere                  icmp echo-request limit: avg 1/sec burst 1
DROP       icmp --  anywhere               anywhere                  icmp echo-request

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination              tcp dpt:telnet
DROP       tcp  --  10.10.10.10            anywhere                  ctstate INVALID
DROP       tcp  --  anywhere               anywhere                  tcp dpt:ssh #conn src/32 > 3
ACCEPT     icmp --  anywhere               anywhere                  icmp echo-request limit: avg 1/sec burst 1
DROP       icmp --  anywhere               anywhere                  icmp echo-request

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination

```

Рисунок 17 – Исправление правил

Блокировка Telnet с 10.10.10.10

Запустим на удаленном сервере

```

sudo python3 -c "import socket; s=socket.socket(); s.bind(('0.0.0.0', 23));
s.listen(1); print('Telnet listening on port 23'); s.accept()"

```

После первого приема он перестанет слушать 23-ий порт, после чего он будет слушать порт 23 (Telnet). Проверим блокировку с двух клиентов. Запускаем на них обоих следующую команду:

```

timeout 5 bash -c '</dev/tcp/192.168.1.20/23' && echo "Port open" || echo
"Port closed".

```

На рисунке 18 представлен скриншот тестирования.

```

Server-1 - PuTTY
root@debian:~#
root@debian:~# sudo python3 -c "import socket; s=socket.socket(); s.bind(('0.0.0.0', 23)); s.listen(1); print('Telnet listening on port 23'); s.accept()"
Telnet listening on port 23
root@debian:~#

InternalClient - PuTTY
debian@debian:~$ timeout 5 bash -c '</dev/tcp/192.168.1.20/23' && echo "Port open" || echo "Port closed"
Port open
debian@debian:~$

ExternalClient - PuTTY
debian@debian:~$
debian@debian:~$ timeout 5 bash -c '</dev/tcp/192.168.1.20/23' && echo "Port open" || echo "Port closed"
Port closed
debian@debian:~$

```

Рисунок 18 – Тестирование блокировки TCP-запросов

Блокировка INVALID TCP пакетов

Нужно создать от лица админа на внешнем клиенте python-файл (nano test_invalid.py)

Листинг 1 – Содержимое файла

```

#!/usr/bin/env python3
import socket
import struct

def send_invalid_ack():
    """Отправляет ACK пакет без установленного соединения"""
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_RAW,
socket IPPROTO_TCP)

```

```

tcp_header = struct.pack('!HLLBBH',
    12345,
    22,
    0, 0,
    5 << 4,
    0x10,
    65535
) + b'\x00\x00'
s.sendto(tcp_header, ('192.168.1.20', 0))
print("INVALID ACK packet sent to 192.168.1.20:22")
s.close()
return True
except PermissionError:
    print("Ошибка: нужны права root (sudo)")
    return False
except Exception as e:
    print(f"Ошибка: {e}")
    return False

if __name__ == "__main__":
    send_invalid_ack()

```

После чего сохраняем (Ctrl+O, Enter, Ctrl+X). Теперь открываем сервер с wirewall и смотрим трафик с помощью команды

```
sudo iptables -L FORWARD -n -v | grep INVALID
```

После чего запускаем файл на клиенте (sudo python3 test_invalid.py) и снова проверяем трафик. Результаты в скриншоте ниже (рис. 19). Счетчик отброшенных пакетов увеличился, а значит, правило работает корректно.

```

ExternalClient - PuTTY
root@debian:~# sudo python3 test_invalid.py
INVALID ACK packet sent to 192.168.1.20:22
root@debian:~#
root@u-live-srvr:~# sudo iptables -L FORWARD -n -v | grep INVALID
4 152 DROP 6 -- * * 0.0.0.0/0 0.0.0.0/0 ctstate INVALID
root@u-live-srvr:~# sudo iptables -L FORWARD -n -v | grep INVALID
5 190 DROP 6 -- * * 0.0.0.0/0 0.0.0.0/0 ctstate INVALID
root@u-live-srvr:~# _

```

Рисунок 19 – Тестирование блокировки TCP пакетов

Лимит SSH соединений

На внешнем (рис. 20) и внутреннем (рис. 21) клиентах запустили одновременно 4 TCP-соединения к порту 22. На внешнем три из них прошли в соответствии с правилом iptables, а одно заблокировалось, на внутреннем – все прошли.

```

[1] 397debian:~$ for i in {1..4}; do (timeout 3 bash -c '</dev/tcp/192.168.1.20/22' && echo "SUCCESS $i" || echo "FAILED $i") & done; wait
[2] 398
[3] 399
[4] 402
SUCCESS 2
SUCCESS 3
SUCCESS 4
SUCCESS 1
[1] Done ( timeout 3 bash -c '</dev/tcp/192.168.1.20/22' && echo "SUCCESS $i" || echo "FAILED $i" )
[2] Done ( timeout 3 bash -c '</dev/tcp/192.168.1.20/22' && echo "SUCCESS $i" || echo "FAILED $i" )
[3] Done ( timeout 3 bash -c '</dev/tcp/192.168.1.20/22' && echo "SUCCESS $i" || echo "FAILED $i" )
[4]+ Done for i in {1..4}; do (timeout 3 bash -c '</dev/tcp/192.168.1.20/22' && echo "SUCCESS $i" || echo "FAILED $i") & done; wait

```

Рисунок 20 – Проверка внутреннего клиента

```

debian@debian:~$ for i in {1..4}; do (timeout 3 bash -c '</dev/tcp/192.168.1.20/22' && echo "SUCCESS $i" || echo "FAILED $i") & done; wait
[1] 409
[2] 410
[3] 411
[4] 412
SUCCESS 4
SUCCESS 3
SUCCESS 2
FAILED 1
[1] Done ( timeout 3 bash -c '</dev/tcp/192.168.1.20/22' && echo "SUCCESS $i" || echo "FAILED $i" )
[2] Done ( timeout 3 bash -c '</dev/tcp/192.168.1.20/22' && echo "SUCCESS $i" || echo "FAILED $i" )
[3] Done ( timeout 3 bash -c '</dev/tcp/192.168.1.20/22' && echo "SUCCESS $i" || echo "FAILED $i" )
[4]+ Done ( timeout 3 bash -c '</dev/tcp/192.168.1.20/22' && echo "SUCCESS $i" || echo "FAILED $i" )
debian@debian:~$

```

Рисунок 21 – Проверка с внешнего клиента

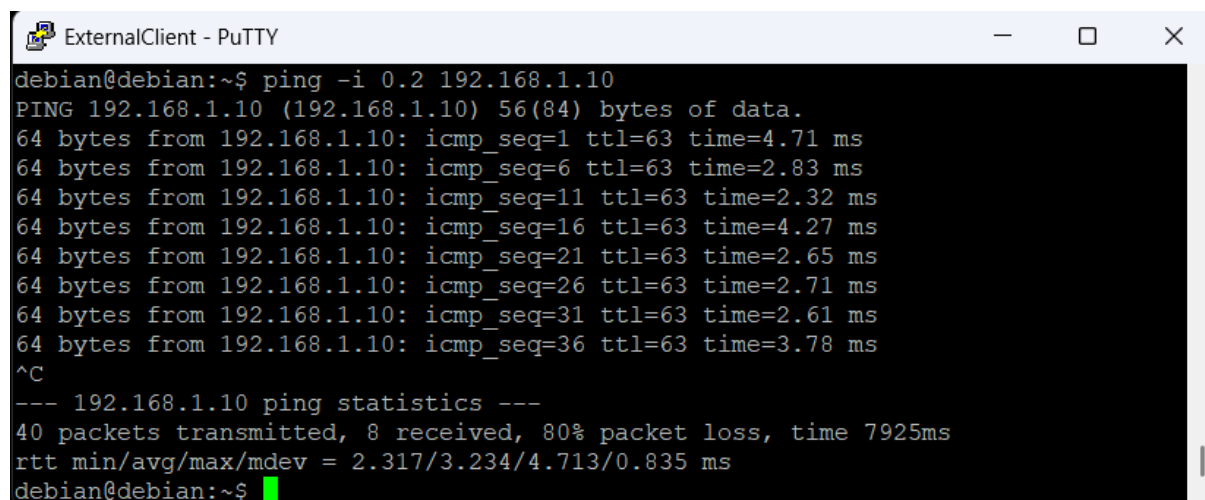
Лимит ICMP (Ping 1 раз в секунду)

На внешнем клиенте запустим пинг до сервера (рис. 22) с помощью команды:

```
ping -i 0.2 192.168.2.10
```

Приходит ответ на каждый 5-ый пакет. Это происходит потому, что мы отправляем ping-запросы каждые 0.2 секунды (интервал 0.2 с = 5 пакетов/сек), а firewall настроен на пропускание не более 1 ICMP echo-request в секунду.

Таким образом, из каждых 5 отправленных пакетов firewall пропускает только 1, а 4 отбрасывает, что подтверждает корректную работу правила ограничения ICMP-трафика.



```

ExternalClient - PuTTY
debian@debian:~$ ping -i 0.2 192.168.1.10
PING 192.168.1.10 (192.168.1.10) 56(84) bytes of data.
64 bytes from 192.168.1.10: icmp_seq=1 ttl=63 time=4.71 ms
64 bytes from 192.168.1.10: icmp_seq=6 ttl=63 time=2.83 ms
64 bytes from 192.168.1.10: icmp_seq=11 ttl=63 time=2.32 ms
64 bytes from 192.168.1.10: icmp_seq=16 ttl=63 time=4.27 ms
64 bytes from 192.168.1.10: icmp_seq=21 ttl=63 time=2.65 ms
64 bytes from 192.168.1.10: icmp_seq=26 ttl=63 time=2.71 ms
64 bytes from 192.168.1.10: icmp_seq=31 ttl=63 time=2.61 ms
64 bytes from 192.168.1.10: icmp_seq=36 ttl=63 time=3.78 ms
^C
--- 192.168.1.10 ping statistics ---
40 packets transmitted, 8 received, 80% packet loss, time 7925ms
rtt min/avg/max/mdev = 2.317/3.234/4.713/0.835 ms
debian@debian:~$

```

Рисунок 22 – Тестирование лимита ICMP

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы был изучен и практически освоен инструмент управления межсетевым экраном iptables в ОС Linux на примере netfilter. Был развернут лабораторный стенд в эмуляторе GNS3, включающий клиентов, межсетевой экран (IPtablesFW) и удаленный сервер, а также настроена маршрутизация транзитного трафика.

В соответствии с вариантом задания №3 на межсетевом экране были успешно реализованы следующие правила фильтрации и ограничения:

- Блокировка Telnet-соединений с заданного адреса 10.10.10.10 — правило отработало корректно, доступ к порту 23 для указанного источника был запрещен как к самому МЭ, так и транзитом.
- Блокировка «невалидных» TCP-пакетов (не принадлежащих ни одному из установленных соединений) — с помощью модуля conntrack и состояния 'INVALID' такие пакеты успешно отбрасывались, что подтверждается увеличением счетчика пакетов в правиле DROP.
- Ограничение параллельных SSH-соединений — правило с модулем connlimit ограничило количество одновременных подключений к 22-му порту с одного IP-адреса до трех. Четвертое соединение блокировалось, при этом с других адресов подключения проходили без ограничений.
- Ограничение частоты ICMP-запросов (echo-request) — с помощью модуля 'limit' было настроено пропускание не более одного ping-запроса в секунду. Тестирование с интервалом 0.2 секунды показало, что из пяти пакетов проходит только один, остальные отбрасываются, что соответствует заданию.

В процессе тестирования был выявлен нюанс работы модуля conntrack: последующие пакеты одного ping-сеанса классифицируются как ESTABLISHED и могут проходить фильтрацию до применения правила с лимитом. Это следует учитывать при построении сложных цепочек правил.

Таким образом, все поставленные задачи выполнены в полном объеме: лабораторный стенд настроен, правила iptables применены и протестированы, результаты задокументированы. В ходе работы были закреплены практические навыки настройки сетевого экрана Linux для защиты от атак типа brute-force (ограничение числа подключений), сканирования портов (блокировка INVALID-пакетов) и ICMP-флуда (ограничение частоты запросов).